

🔗 Gerando uma Frase Semente BIP39 com Dados de 20 Faces — Edição Estendida: 12 e 24 Palavras

Uma frase semente mnemônica BIP39 fornece a entropia mestre para sua carteira de Bitcoin. Embora softwares possam gerar essa aleatoriedade, o uso de dados físicos oferece uma fonte de entropia transparente e verificável, que não depende de nenhum gerador de números aleatórios eletrônico. Esta edição estendida cobre o procedimento completo de 12 palavras, o procedimento de 24 palavras e o atalho de autocompletar a última palavra oferecido por várias ferramentas de semente. Usa a mesma tabela de consulta (verso).

📖 Por que entropia verificável? — o incidente de julho de 2026

Em julho de 2026, mais de mil BTC foram varridos em minutos de carteiras cujas sementes haviam sido criadas por dispositivos Coldcard com um defeito de firmware existente desde 2021: a geração da semente usava silenciosamente um PRNG previsível de software em vez do RNG de hardware, entregando muito menos entropia do que o prometido. O fabricante corrigiu rapidamente — e o próprio aviso oficial registra que sementes geradas com 50 ou mais rolagens honestas de dados nunca estiveram em risco. A lição não é sobre uma marca: **toda entropia eletrônica é invisível para o usuário**. Não dá para olhar um chip e enxergar se sua aleatoriedade está saudável — mas dá para olhar um dado. Neste tutorial, cada bit da sua semente vem de rolagens que você confere com os próprios olhos.

📊 Entendendo a Estrutura da Tabela

A tabela de referência está organizada como um sistema de consulta tridimensional. Cada uma das 2048 palavras do padrão BIP39 é identificada de forma única por três coordenadas, que chamaremos de D1, D2 e D3. A primeira coordenada (**D1**) seleciona uma das **8 seções** de página, a segunda (**D2**) seleciona uma das **16 linhas** dentro dessa seção, e a terceira (**D3**) seleciona uma das **16 colunas**. Juntos, esses três valores identificam exatamente uma palavra na tabela. As palavras da semente são sempre as do padrão BIP39 em inglês, aceitas por praticamente todas as carteiras.

🎲 O Procedimento de Rolagem (12 Palavras)

Para cada uma das primeiras 11 palavras da sua frase semente, você deve obter três resultados válidos do dado. Role o dado e registre o resultado apenas se ele mostrar um valor entre 1 e 16 (inclusive). Se o dado mostrar 17, 18, 19 ou 20, **descarte essa rolagem** e role novamente até obter um resultado válido. Repita esse processo até ter três números válidos (D1, D2, D3) para a palavra atual.

💡 Exemplo de Geração de Palavra

Suponha que você esteja gerando sua quinta palavra da semente. Você rola o dado e obtém: 19 (inválido, role novamente), 3 (válido, D1 = 3,4), 1 (válido, D2 = 1), 20 (inválido, role novamente), 11 (válido, D3 = 11). Suas coordenadas são ((3,4), 1, 11). Navegue até a seção 3,4 na tabela, encontre a linha 1 e então a coluna 11 para obter sua palavra: **candy**.

✍️ 12ª Palavra

Para gerar a última palavra da sua semente BIP39, repita o procedimento para as 2 primeiras coordenadas igual ao realizado para as 11 primeiras palavras, selecionando, portanto, a seção e a linha da tabela. Neste ponto, existe exatamente uma única palavra na linha selecionada (com 16 alternativas) que formará uma semente BIP39 válida. A determinação da coluna para a última palavra **não** será feita por rolagem de dados, e sim por tentativa e erro no wizard de input da semente da sua carteira — de preferência em um dispositivo *air gapped*: ele rejeitará as 15 candidatas inválidas e aceitará a única válida.

💡 Exemplo de Geração da Última Palavra

Suponha que suas 11 primeiras palavras sejam, em ordem: “never, use, this, example, because, private, key, secret, need, phrase, e true”.

Agora você gerará sua 12ª palavra da semente. Você rola o dado e obtém: 18 (inválido, role novamente), 12 (válido, D1 = 11,12), 9 (válido, D2 = 9). Com isso, a sua 12ª palavra já está determinada: teste uma por uma as palavras da linha 9 da seção 11,12 e você verá que a palavra da coluna 14, **random**, (e apenas ela), formará, juntamente com as 11 primeiras, nesta ordem, uma semente BIP39 válida:

1.never 2.use 3.this 4.example 5.because 6.private 7.key 8.secret 9.need 10.phrase 11.true 12.random. 🗳️

Conferência (p.ex. em ferramenta de autocompletar): palavras válidas da seção 11,12, uma por linha (1–16, nesta ordem): payment, people, planet, pole, possible, prize, pulp, quality, random, raw, relief, result, return, room, royal, say.

🔗 Atalho: Wizards com Autocompletar da Última Palavra

Várias ferramentas de semente completam a última palavra para você: dadas todas as palavras anteriores, elas listam exatamente as candidatas que tornam o *checksum* válido — por exemplo o fluxo *final word* do SeedSigner, a calculadora de última palavra do SeedPicker, o BIP39 Recoverer offline da Coinplate ou a página BIP39 de Ian Coleman (usada offline). Para uma semente de 12 palavras existem sempre exatamente **128 últimas palavras válidas: precisamente uma em cada uma das $8 \times 16 = 128$ linhas** da tabela. Então, em vez da tentativa e erro coluna por coluna: role D1 e D2 como de costume e tome, entre as candidatas do wizard, **a que estiver na sua seção e linha roladas**. Mesma palavra, mesma entropia, menos digitação. E não é um acaso do nosso exemplo: *quaisquer* que sejam as 11 primeiras palavras, fixados D1 e D2, uma e apenas uma palavra daquela linha satisfaz o checksum — os testes de sanidade do repositório verificam isso mecanicamente sobre prefixos aleatórios.

24 Palavras

Para uma semente de 24 palavras (256 bits de entropia), gere as palavras 1–23 exatamente como as 11 primeiras acima: três rolagens válidas cada ($23 \times 11 = 253$ bits). Para a **24ª palavra, role apenas D1**, selecionando a seção (mais 3 bits — 256 no total). A linha e a coluna ficam então fixadas pelo *checksum* de 8 bits: **exatamente uma palavra em toda a sua seção de 256 palavras** completa uma semente válida. Encontre-a de uma de duas formas:

- **Tentativa e erro** no wizard da carteira, ao longo da seção rolada (até 256 tentativas — tedioso, mas totalmente offline); ou
- **Autocompletar da última palavra**: um wizard como os acima oferecerá exatamente **8 candidatas válidas — uma por seção**. Sua rolagem de D1 escolhe a seção; tome a candidata que estiver nela.

Igualmente garantido, não coincidência: *quaisquer* que sejam as 23 primeiras palavras, fixado D1, uma e apenas uma palavra daquela seção satisfaz o checksum — também verificado por máquina pelos testes de sanidade do repositório.

Exemplo de 24 Palavras: a Frase do Exemplo, Duplicada

Pegue o exemplo de 12 palavras acima e duplique-o para obter as palavras 1–23; para a 24ª, suponha que você role: 17 (inválido, role novamente), 11 (válido, D1 = 11,12). Entre as 8 candidatas do wizard (ou por tentativa e erro ao longo da seção 11,12), exatamente uma palavra dessa seção — **polar**, linha 4, coluna 12 — completa uma semente válida de 24 palavras:

```
never use this example because private key secret need phrase true random
never use this example because private key secret need phrase true polar ✓
```

Note o sacrifício: a duplicação não pode manter **random** como 24ª palavra, porque o checksum de 8 bits dos 256 bits completos difere do checksum de 4 bits que tornava **random** válida com 12 palavras. O checksum reivindica a última palavra — e cai em **polar**, a única palavra da *mesma seção* (D1 = 11,12) de **random** cuja combinação com as 23 anteriores o satisfaz. A lista completa de candidatas, uma por seção: **because** (1,2), **critic** (3,4), **fresh** (5,6), **ice** (7,8), **limb** (9,10), **polar** (11,12), **spare** (13,14), **wolf** (15,16).

Resumo

- Role um dado de 20 faces. **Descarte** resultados **17, 18, 19** ou **20**
- Cada rolamento válido (de 1 a 16 inclusive) especifica uma coordenada de uma palavra
- Cada uma das **8 seções (D1)** corresponde a **2** resultados válidos possíveis; cada uma das **16 linhas (D2)** e **16 colunas (D3)**, a **1**
- **12 palavras**: 11 palavras roladas por completo e depois apenas D1 + D2; **35 rolagens válidas** produzem os **128 bits** completos. A última palavra: tentativa e erro na linha, ou a candidata do wizard na sua linha rolada (existem **128** candidatas válidas, uma por linha)
- **24 palavras**: 23 palavras roladas por completo e depois apenas D1; **70 rolagens válidas** produzem os **256 bits** completos. A última palavra: tentativa e erro na seção, ou a candidata do wizard na sua seção rolada (existem **8** candidatas válidas, uma por seção)
- Os bits de *checksum* (4 ou 8) são sempre por conta da carteira — nunca rolados

Avisos de Segurança

- Nunca use este exemplo acima porque chaves privadas, para serem secretas, precisam que sua frase seja verdadeiramente randômica
- Realize este procedimento em um local privado. Não fotografe nem registre digitalmente suas rolagens ou resultados intermediários. Escreva sua frase semente final em papel e guarde-a em local seguro
- Só use ferramentas de autocompletar em dispositivos *air gapped* de confiança — elas veem a sua semente inteira. Para máxima segurança, apenas digite sua frase semente em dispositivos *air gapped* de sua confiança e propriedade
- Se você suspeita que uma semente sua foi gerada por software ou firmware defeituoso (como no incidente de julho de 2026), gere uma semente nova com este método e transfira seus fundos para ela: atualizar o firmware **não** conserta uma semente fraca já existente

Saiba Mais

Este tutorial é de autoria de **Yuri da Silva Villas Boas** — autor da BIP-450 (Formosa) e criador do protocolo Great Wall. Repositório oficial (fontes, PDFs em 4 idiomas, verificação automática): github.com/Yuri-SVB/BTC-D20 — ou escaneie o QR code ao lado.

Artigo curto: por que um aviso discreto era impossível — Kerckhoffs e o incidente de julho de 2026: [posts/kerckhoffs-lemma-coldcard.mdno](https://posts.kerckhoffs-lemma-coldcard.mdno) repositório

Mais tutoriais, cursos e comunidade: yuri-svb.github.io — e siga-nos no Twitter/X, Nostr e Telegram

Great Wall: Gerar uma semente forte é só o primeiro passo; protegê-la contra roubo e coerção é o resto. Great Wall é nosso protocolo de software livre para autocustódia resistente a coerção: github.com/Yuri-SVB

